Standard and Grover-Mixer QAOA for Generalized Bipartite Assignment

A Reproducible QUBO Benchmark on IBM Quantum Hardware

Iván Barrientos Salas* Ilia Fazeli Jiya Maurya Marcos Jiménez
Abhishek Raj Satkar Juneja

QI26-20 / QIntern 2026 Research Team

August 2026

Abstract

This work presents a reproducible benchmark of standard Quantum Approximate Optimization Algorithm (QAOA) and Grover-Mixer QAOA (GM-QAOA) for generalized bipartite assignment. The benchmark preserves the original student-developed five-track workflow and extends it to three fixed instances: a 3×3 one-to-one assignment, a reproducible 4×4 one-to-one assignment, and a 4×2 exact-capacity assignment with $\boldsymbol{C} = (2, 2)$. Standard QAOA is evaluated in ideal statevector simulation for depths $p \in \{1, 2, 3\}$, whereas GM-QAOA is evaluated at $p = 1$ using exact small-instance feasible-state preparations. The two $p = 1$ circuits for the capacitated instance are then transpiled with the same preset pass manager and executed in one IBM Quantum Runtime job on `ibm_marrakesh`, using 1024 shots per circuit, dynamical decoupling, and gate twirling.

Both hardware distributions contained the classical optimum of score 3.25, yielding zero best-sample optimization gap. However, distribution quality differed sharply from that conclusion alone. Standard QAOA produced an observed feasible probability of 7.7148% and an optimal-state probability of 1.5625%, compared with ideal values of 9.1582% and 2.3380%. GM-QAOA produced 11.9141% feasible mass and 2.8320% optimal-state probability, compared with ideal values of 100% and 50.3666%. The corresponding total variation distances were approximately 0.2082 for standard QAOA and 0.8809 for GM-QAOA. Transpilation generated depth 143 and 63 native CZ gates for standard QAOA, versus depth 1357 and 496 CZ gates for GM-QAOA. Thus, the implemented Grover mixer satisfies the ideal feasible-subspace construction of Bärtschi and Eidenbenz for the fixed instances, but its hardware realization incurs substantial routing and synthesis overhead. The study establishes a transparent small-instance benchmark; it does not establish quantum advantage or a general scalable state-preparation method.

Keywords: QAOA; GM-QAOA; QUBO; generalized bipartite assignment; exact capacities; IBM Quantum; transpilation; NISQ noise.

*Corresponding author and QI26-20 mentor.

Contents

List of Figures

List of Tables

1 Introduction and Motivation

Combinatorial assignment is a recurring primitive in logistics, scheduling, enterprise resource allocation, and matching systems. A typical instance links a set of agents to a set of resources through an affinity or utility matrix, while enforcing exclusivity or capacity constraints. Even when each local score is easy to evaluate, the number of globally admissible assignments grows combinatorially. The QI26-20 project therefore frames generalized bipartite assignment as a Quadratic Unconstrained Binary Optimization (QUBO) problem and benchmarks quantum variational optimization against exact and heuristic classical methods [1].

Current quantum processors operate in the noisy intermediate-scale quantum (NISQ) regime. For such hardware, an algorithm cannot be evaluated only by whether an optimal bitstring appears at least once. The probability mass assigned to feasible states, the concentration on optimal assignments, circuit depth, two-qubit-gate count, and the discrepancy between ideal and physical distributions are all necessary to interpret a result. This is especially important for constrained optimization, where standard QAOA explores the full computational basis while penalty terms energetically discourage infeasible states.

The Quantum Alternating Operator Ansatz introduced a broader framework for constrained optimization in which the initial state and mixer can be adapted to the feasible subspace [2]. Bärtschi and Eidenbenz proposed GM-QAOA, which prepares an equal superposition of all feasible solutions and uses a Grover-like selective phase-shift mixer [3]. In ideal evolution, this construction remains entirely within the feasible subspace and assigns equal amplitudes to feasible solutions having equal objective values. Its central trade-off is equally important: mixer design becomes simple only when the feasible-state preparation unitary U_S is efficient.

The present study extends the original 3×3 QI26-20 benchmark with two controlled instances requested by the depth supplement: a 4×4 one-to-one problem and a 4×2 exact-capacity problem with $\boldsymbol{C} = (2, 2)$. The standard-QAOA analysis is restricted to the requested depths $p \in \{1, 2, 3\}$ and uses one deterministic optimization per configuration. The hardware comparison uses the compact 4×2 instance and $p = 1$ for both standard QAOA and GM-QAOA.

1.1 Research questions

The benchmark addresses the following questions:

1. How do feasible and optimal-solution probabilities vary with standard-QAOA depth $p \in \{1, 2, 3\}$ across one-to-one and exact-capacity instances?
2. Does the implemented GM-QAOA circuit reproduce the theoretical feasible-subspace evolution in ideal simulation?
3. How far do the measured IBM Quantum distributions deviate from their ideal references?
4. What physical overhead is introduced by mapping standard QAOA and the Grover-mixer construction to the same backend?
5. Which requirements of QI26-20 and Bärtschi–Eidenbenz are supported by the artifact, and which claims remain outside its scope?

1.2 Contributions

The main contributions are:

- a programmatic QUBO model supporting both one-to-one assignments and exact capacity vectors $\boldsymbol{C}$;
- a reproducible comparison of three fixed instances and four classical baselines;

- a deterministic ideal standard-QAOA depth study for $p = 1, 2, 3$;
- exact small-instance GM-QAOA preparations and circuit-level validation of U_S, U_P, and U_M;
- a controlled IBM Quantum experiment in which both algorithms share the same backend, shots, pass manager, and runtime job;
- full-distribution error metrics, feasible-subspace leakage, statistical intervals, and transpilation-resource analysis;
- an explicit compliance matrix and claims boundary suitable for an open research repository.

1.3 Scope of the claims

The work demonstrates a reproducible small-instance implementation. It does not demonstrate asymptotic quantum speedup, practical runtime advantage, or the general $O(n^3)$-gate permutation-state preparation described in the reference paper. Hardware conclusions are tied to one backend, one calibration period, one job, one exact-capacity instance, and $p = 1$.

2 Assignment Problem Definition

2.1 Sets, variables, and objective

Let

$$\mathcal{A} = \{0, \ldots, n-1\}, \qquad \mathcal{R} = \{0, \ldots, m-1\} \tag{1}$$

be disjoint sets of agents and resources. The binary decision variable

$$x_{ij} = \begin{cases} 1, & \text{if agent } i \text{ is assigned to resource } j, \\ 0, & \text{otherwise} \end{cases} \tag{2}$$

is defined for every $(i, j) \in \mathcal{A} \times \mathcal{R}$. Given an affinity matrix $S \in \mathbb{R}^{n \times m}$, the classical objective is

$$\max_x f(x) = \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} S_{ij} x_{ij}. \tag{3}$$

Every supported instance enforces exactly one resource per agent:

$$\sum_{j=0}^{m-1} x_{ij} = 1, \qquad \forall i \in \mathcal{A}. \tag{4}$$

2.2 One-to-one assignment

For one-to-one matching, every resource is used at most once:

$$\sum_{i=0}^{n-1} x_{ij} \leq 1, \qquad \forall j \in \mathcal{R}. \tag{5}$$

When $n = m$, Eqs. (4)–(5) define permutation matrices.

2.3 Exact-capacity assignment

For the generalized case, each resource receives a prescribed integer capacity:

$$\boldsymbol{C} = (C_0, \ldots, C_{m-1}), \qquad C_j \in \mathbb{Z}_{\geq 0}, \qquad \sum_{j=0}^{m-1} C_j = n, \tag{6}$$

and the resource constraints are

$$\sum_{i=0}^{n-1} x_{ij} = C_j, \qquad \forall j \in \mathcal{R}. \tag{7}$$

The exact-capacity model is the fixed one-to-N extension used in this phase.

2.4 Feasible set

For an instance $I = (S, \boldsymbol{C}, \text{mode})$, let $\mathcal{F}_I$ denote the set of binary matrices satisfying Eq. (4) and the applicable resource constraints. The cardinalities of the three fixed feasible sets are

$$|\mathcal{F}_{3\times3}| = 3! = 6, \qquad |\mathcal{F}_{4\times4}| = 4! = 24, \qquad |\mathcal{F}_{4\times2}| = \binom{4}{2} = 6. \tag{8}$$

2.5 Benchmark instances

The fixed matrices are

$$S^{(3\times3)} = \begin{pmatrix} 0.90 & 0.20 & 0.40 \\ 0.30 & 0.85 & 0.50 \\ 0.45 & 0.35 & 0.95 \end{pmatrix}, \qquad \boldsymbol{C} = (1,1,1), \tag{9}$$

$$S^{(4\times4)} = \begin{pmatrix} 0.29 & 0.66 & 0.52 & 0.45 \\ 0.43 & 0.78 & 0.87 & 0.29 \\ 0.67 & 0.39 & 0.92 & 0.89 \\ 0.66 & 0.75 & 0.56 & 0.81 \end{pmatrix}, \qquad \boldsymbol{C} = (1,1,1,1), \tag{10}$$

$$S^{(4\times2)} = \begin{pmatrix} 0.85 & 0.30 \\ 0.40 & 0.90 \\ 0.70 & 0.55 \\ 0.25 & 0.80 \end{pmatrix}, \qquad \boldsymbol{C} = (2,2). \tag{11}$$

The 4×4 matrix is generated by NumPy's default random generator with seed 2026 and uniform range $[0.15, 0.95]$, rounded to two decimals. The 3×3 and 4×2 matrices are explicit reference instances.

For the capacitated instance, the optimal assignment is

$$X^\star = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad f(X^\star) = 0.85 + 0.90 + 0.70 + 0.80 = 3.25. \tag{12}$$

With the notebook's little-endian row-major convention, this matrix is reported as bitstring `10011001`.

3 QUBO Formulation

3.1 One-to-one objective with penalties

The maximization in Eq. (3) is represented as minimization. For the one-to-one case, the implemented energy is

$$Q_{1:1}(x) = -\sum_{i,j} S_{ij}x_{ij} + \lambda_A \sum_i \left(\sum_j x_{ij} - 1\right)^2 + \lambda_R \sum_j \sum_{i<k} x_{ij}x_{kj}. \tag{13}$$

The first penalty enforces exactly one assignment in each agent row. The second penalizes resource collisions. In square instances, the combination yields permutation matrices at the feasible minimum.

3.2 Exact-capacity QUBO

For a capacity vector $\boldsymbol{C}$, the resource-side penalty becomes

$$Q_C(x) = -\sum_{i,j} S_{ij}x_{ij} + \lambda_A \sum_i \left(\sum_j x_{ij} - 1\right)^2 + \lambda_R \sum_j \left(\sum_i x_{ij} - C_j\right)^2. \tag{14}$$

This exact squared penalty is the minimal generalization requested in the depth supplement [4].

3.3 Penalty scale and coefficient expansion

The student-developed deterministic scale is retained:

$$\lambda_A = \lambda_R = nmS_{\max}, \qquad S_{\max} = \max_{i,j} |S_{ij}|. \tag{15}$$

For the three instances this gives Table 1.

Table 1: Penalty weights and constant offsets.

Instance	$N = nm$	λ_A	λ_R	Constant offset
3×3 one-to-one	9	8.55	8.55	25.65
4×4 one-to-one	16	14.72	14.72	58.88
4×2, $\boldsymbol{C} = (2, 2)$	8	7.20	7.20	86.40

Writing a generic upper-triangular QUBO as

$$Q(x) = c + \sum_q a_q x_q + \sum_{q<r} b_{qr}x_q x_r, \tag{16}$$

the exact-capacity expansion yields

$$c = n\lambda_A + \lambda_R \sum_j C_j^2, \tag{17}$$

$$a_{ij} = -S_{ij} - \lambda_A + \lambda_R(1 - 2C_j), \tag{18}$$

$$b_{(i,j),(i,\ell)} = 2\lambda_A, \qquad j < \ell, \tag{19}$$

$$b_{(i,j),(k,j)} = 2\lambda_R, \qquad i < k. \tag{20}$$

The coefficients are produced programmatically; no manually typed QUBO matrix is used in the quantum stage.

3.4 QUBO-to-Ising mapping

Each binary variable is mapped to a Pauli-Z operator through

$$x_q = \frac{1 - Z_q}{2}, \qquad x_q x_r = \frac{I - Z_q - Z_r + Z_q Z_r}{4}. \tag{21}$$

Thus,

$$H_Q = \alpha I + \sum_q h_q Z_q + \sum_{q<r} J_{qr} Z_q Z_r, \tag{22}$$

where

$$\alpha = c + \frac{1}{2}\sum_q a_q + \frac{1}{4}\sum_{q<r} b_{qr}, \tag{23}$$

$$h_q = -\frac{1}{2}a_q - \frac{1}{4}\sum_{r\neq q} b_{qr}, \tag{24}$$

$$J_{qr} = \frac{1}{4} b_{qr}. \tag{25}$$

The notebook constructs the operator with explicit sparse Pauli terms and verifies basis-state equivalence between $Q(x)$ and $\langle x|H_Q|x\rangle$. The validation is exhaustive for all 512 states of the 3×3 instance and all 256 states of the 4×2 instance; the 4×4 case is checked on deterministic reference states, optimum assignments, and control samples.

3.5 Logical interaction topology

The quadratic row and column couplings produce a dense rook-graph structure over the one-hot variables. Figure 1 shows the logical graphs for the two square instances. This density is relevant because the IBM hardware coupling graph is sparse; routing therefore introduces SWAP-equivalent overhead even when the logical qubit count remains modest.

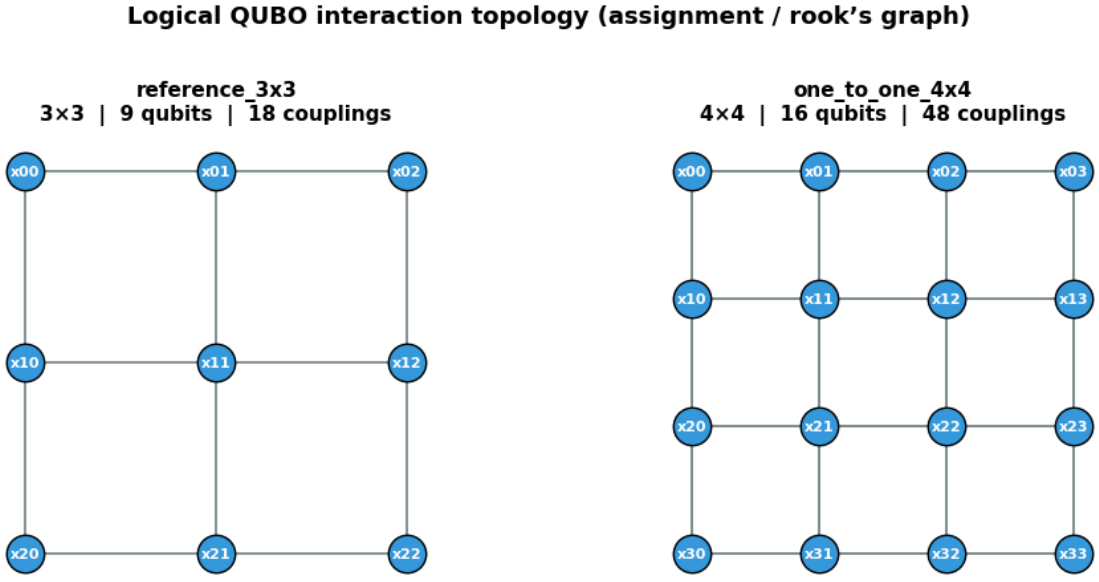

Figure 1: Logical QUBO interaction topology retained from the executed notebook. Row and column cliques create dense couplings before hardware mapping.

4 Classical Baselines

The benchmark uses classical solvers on the same matrices and constraints as the quantum workflows. Every solver returns an assignment matrix, score, feasibility flag, runtime, and gap relative to the exact optimum.

4.1 Exact enumeration

For the fixed small instances, exact enumeration provides the reference optimum. One-to-one assignments are enumerated as injections or permutations. Exact-capacity assignments are enumerated over resource labels and retained only when their histogram equals $\boldsymbol{C}$. This method is exponential in general but unambiguous for the present validation sizes.

4.2 Hungarian algorithm

The Hungarian algorithm solves the square one-to-one linear assignment problem in polynomial time [5]. Because the benchmark maximizes affinity, the implementation converts the score matrix to the corresponding minimization form. It is not applied directly to the 4×2 exact-capacity representation.

4.3 Greedy heuristic

The greedy solver selects high-affinity pairs while respecting the active resource constraints. It is computationally inexpensive but locally myopic; its failure on the 4×4 instance is useful as a non-optimal reference.

4.4 Simulated annealing

Simulated annealing uses stochastic local updates and an acceptance schedule to explore the assignment space [6]. The notebook fixes seed 2026 and evaluates feasibility using the same validation functions as the exact and quantum candidates.

4.5 Evaluation quantities

For maximization, the absolute and relative gaps are

$$\Delta_{\mathrm{abs}} = f^\star - f_{\mathrm{best}}, \qquad \Delta_{\mathrm{rel}} = \frac{f^\star - f_{\mathrm{best}}}{|f^\star|}. \tag{26}$$

A gap of zero indicates that the best returned feasible assignment equals the exact score. It does not by itself describe sampling concentration or expected energy.

5 QAOA and GM-QAOA Implementation

5.1 Standard QAOA

Standard QAOA follows the alternating-operator construction introduced by Farhi, Goldstone, and Gutmann [7] and begins in the uniform superposition

$$|+\rangle^{\otimes N} = H^{\otimes N}|0\rangle^{\otimes N} \tag{27}$$

and alternates the penalized cost Hamiltonian H_Q with the transverse-field mixer

$$H_M = \sum_{q=0}^{N-1} X_q. \tag{28}$$

At depth p, the variational state is

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = \prod_{\ell=1}^{p} e^{-i\beta_\ell H_M} e^{-i\gamma_\ell H_Q} |+\rangle^{\otimes N}. \tag{29}$$

The outer loop minimizes $\langle H_Q \rangle$ with COBYLA. The depth study uses one deterministic initialization and one optimization for each instance–depth pair, rather than a grid search or multi-seed campaign.

5.2 Grover-Mixer QAOA

Let $\mathcal{F}$ be the feasible set. GM-QAOA assumes a unitary U_S satisfying

$$U_S|0\rangle^{\otimes N} = |F\rangle = \frac{1}{\sqrt{|\mathcal{F}|}} \sum_{x \in \mathcal{F}} |x\rangle. \tag{30}$$

The phase separator is diagonal:

$$U_P(\gamma) = e^{-i\gamma H_C}. \tag{31}$$

Inside $\mathcal{F}$, every constraint penalty is zero, so the implemented H_C contains only the assignment objective,

$$H_C = -\sum_{i,j} S_{ij} x_{ij}. \tag{32}$$

The Grover-like mixer is

$$U_M(\beta) = e^{-i\beta |F\rangle\langle F|} \tag{33}$$

$$= U_S \left[I - \left(1 - e^{-i\beta}\right) |0\rangle\langle 0| \right] U_S^\dagger. \tag{34}$$

Consequently, a p-round state has the form

$$|\boldsymbol{\beta}, \boldsymbol{\gamma}\rangle = U_M(\beta_p) U_P(\gamma_p) \cdots U_M(\beta_1) U_P(\gamma_1) U_S |0\rangle^{\otimes N}. \tag{35}$$

The theoretical construction remains inside $\mathcal{F}$, and equal-quality feasible solutions receive equal amplitudes [3].

5.3 Fixed feasible-state preparations

The Phase-1 implementation constructs exact preparations for three instances:

- the uniform superposition of six 3×3 permutation matrices;
- the uniform superposition of 24 4×4 permutation matrices;
- the uniform superposition of six 4×2 assignments with two agents per resource.

The 4×2 preparation creates a weight-two state over the four variables associated with resource 0 and computes the complementary resource-1 variables. Circuit and statevector fidelity are verified against the direct feasible-subspace calculation.

5.4 Physical realization of the Grover mixer

For each round, the circuit applies local objective phases, $U_S^\dagger$, X gates on every problem qubit, a multi-controlled phase gate, the inverse X layer, and U_S. This directly implements Eq. (34); it avoids Trotterization error in the logical mixer. It does not eliminate hardware errors, and the cost of decomposing U_S, $U_S^\dagger$, and the multi-controlled phase can be large.

Bärtschi and Eidenbenz give an explicit general permutation preparation with $O(n^3)$ gates and depth $O(n^2)$ on linear-nearest-neighbor architectures [3]. The present fixed preparations are exact

small-instance constructions, not an implementation of that asymptotic generator. This distinction is maintained throughout the results and compliance analysis.

6 Experimental Setup

6.1 Software environment

The repository uses the Qiskit software stack [8] and pins Qiskit 2.5.1, Qiskit Aer 0.17.2, Qiskit IBM Runtime 0.48.0, NumPy 2.x, pandas 2.x, SciPy 1.14 or newer, Matplotlib 3.8 or newer, and NetworkX 3.2 or newer. The notebook is designed for sequential execution in Google Colab or an equivalent Python environment. COBYLA is provided through SciPy [9].

The random seed for data generation, statevector sampling, and transpilation is 2026. Standard-QAOA candidate tables use 4096 ideal shots. The hardware comparison uses 1024 shots per circuit.

6.2 Ideal depth protocol

Standard QAOA is optimized for

$$p \in \{1, 2, 3\} \tag{36}$$

on all three instances. The original 3×3, $p = 1$ experiment is retained. No additional parameter grid, multiple optimization seeds, or depth values are introduced. GM-QAOA is optimized at $p = 1$ for each fixed instance.

6.3 IBM Quantum execution

The physical experiment targets the 4×2 exact-capacity instance and uses $p = 1$ for both algorithms. The archived job metadata are summarized in Table 2.

Table 2: IBM Quantum job metadata.

Field	Recorded value
Backend	`ibm_marrakesh`
Backend qubits	156
Job identifier	`d9vl8qfo3ppc73akfak0`
Created (UTC)	2026-08-14 17:46:17.684431
Shots per circuit	1024
Circuits in job	2
Twirling realizations	16 per circuit
Shots per realization	64
Dynamical decoupling	Enabled, XpXm
Gate twirling	Enabled
Estimated running time	5.075 875 s
Recorded execution span	4.400 754 s
Client wall time	13.695 772 s

The execution span is derived from Runtime timestamps. Client wall time includes submission, service communication, and result retrieval and is not an algorithm-specific QPU duration.

6.4 Transpilation

Both measured circuits use the same backend-specific preset pass manager:

Listing 1: Backend-specific compilation used in the notebook.

```
pass_manager = generate_preset_pass_manager(
    backend=target_backend,
    optimization_level=3,
    seed_transpiler=2026,
)
standard_isa = pass_manager.run(standard_measured)
gm_isa = pass_manager.run(gm_measured)
```

This process selects a physical layout, decomposes non-native gates, and routes interactions through the backend coupling graph. The final native two-qubit operation in the recorded circuits is CZ; the exported CX and ECR counts are zero.

Figure 2 summarizes the logical-to-physical mismatch visible in the executed notebook.

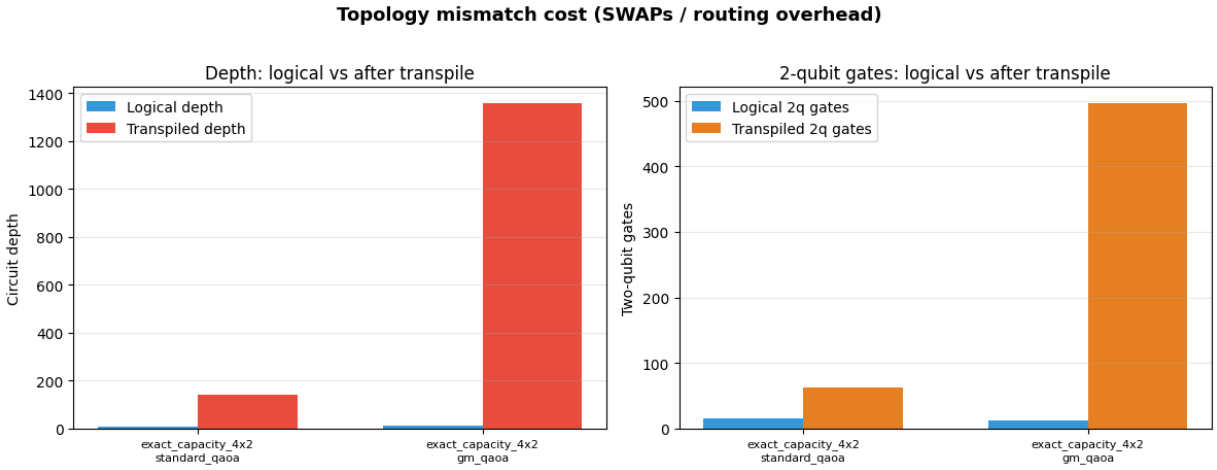

Figure 2: Topology and transpilation-overhead summary generated by the executed notebook. Dense logical assignment couplings must be routed on a sparse device graph.

6.5 Distribution and uncertainty metrics

For ideal distribution p and measured distribution q,

$$\mathrm{TVD}(p,q) = \frac{1}{2}\sum_x |p(x) - q(x)|, \tag{37}$$

$$F_{\mathrm{cl}}(p,q) = \left(\sum_x \sqrt{p(x)q(x)}\right)^2, \tag{38}$$

$$H(p,q) = \sqrt{1 - \sum_x \sqrt{p(x)q(x)}}. \tag{39}$$

F_{cl} is a classical fidelity between measurement distributions, not full quantum-state fidelity on the device. Feasible probability, optimal probability, and leakage are

$$P_{\mathrm{F}} = \sum_{x\in\mathcal{F}} q(x), \qquad P_{\mathrm{opt}} = \sum_{x:f(x)=f^\star} q(x), \qquad L_{\mathcal{F}} = 1 - P_{\mathrm{F}}. \tag{40}$$

Wilson 95% intervals are used for binomial feasible and optimal counts [10].

6.6 Reconstruction of the complete analysis distribution

The archived Runtime job contains the complete 1024-shot BitArray for each circuit, from which the full counts are decoded. The GM-QAOA ideal distribution is exactly reproduced from its exported β and γ. The repository exports standard-QAOA ideal probabilities to six decimal places but does not retain the full ideal distribution as a separate file. For global standard-QAOA distances, the analytical $p = 1$ state is reconstructed by matching those exported probabilities and the exported expected energy, feasible probability, and optimal probability. The recovered values are $\beta \approx 1.8352733$ and $\gamma \approx 1.2267104$. Standard global distances are therefore reported to four significant digits and identified as reconstructed within the repository's export precision.

7 Results

7.1 Classical baseline results

Table 3 reports all classical runs. Exact enumeration and the Hungarian algorithm recover the optimum whenever applicable. Greedy and simulated annealing expose a nontrivial 4×4 case: greedy stops at score 2.80, and simulated annealing at 3.01, versus the exact value 3.08.

Table 3: Classical baseline results exported by the repository.

Instance	Solver	Score	Gap	Runtime (s)
3×3	Exact	2.70	0.00	0.000466
	Hungarian	2.70	0.00	0.000203
	Greedy	2.70	0.00	0.003466
	Simulated annealing	2.70	0.00	1.909880
4×4	Exact	3.08	0.00	0.003566
	Hungarian	3.08	0.00	0.000151
	Greedy	2.80	0.28	0.000417
	Simulated annealing	3.01	0.07	2.707101
4×2, $\boldsymbol{C} = (2, 2)$	Exact	3.25	0.00	0.000305
	Greedy	3.25	0.00	0.000309
	Simulated annealing	3.25	0.00	1.106859

These runtimes make clear that the quantum experiment is a correctness and NISQ implementation benchmark, not a runtime competition against classical optimization at this problem size.

7.2 Standard-QAOA depth comparison

The ideal depth results are given in Table 4. Bar height in Figure 3 represents feasible probability, while labels report optimal probability.

Table 4: Ideal standard-QAOA depth sweep.

Instance	p	$\langle Q \rangle$	P_{F} (%)	P_{opt} (%)	Best score	Gap	Logical depth	Runtime (s)
3×3	1	39.226215	0.3583	0.0740	2.70	0.00	21	1.676498
	2	15.257437	8.9024	2.1687	2.70	0.00	41	1.119079
	3	21.386914	11.8618	4.1527	2.70	0.00	61	1.650187
4×4	1	42.062530	0.2523	0.0029	2.80	0.28	33	12.776456
	2	83.253245	3.9986	0.1629	3.08	0.00	59	15.722948
	3	87.475826	0.0978	0.0092	2.80	0.28	85	16.660130
4×2	1	18.344321	9.1582	2.3380	3.25	0.00	21	0.297405
	2	8.953983	34.8893	5.4127	3.25	0.00	38	0.358492
	3	7.934315	43.9911	6.1093	3.25	0.00	55	0.452604

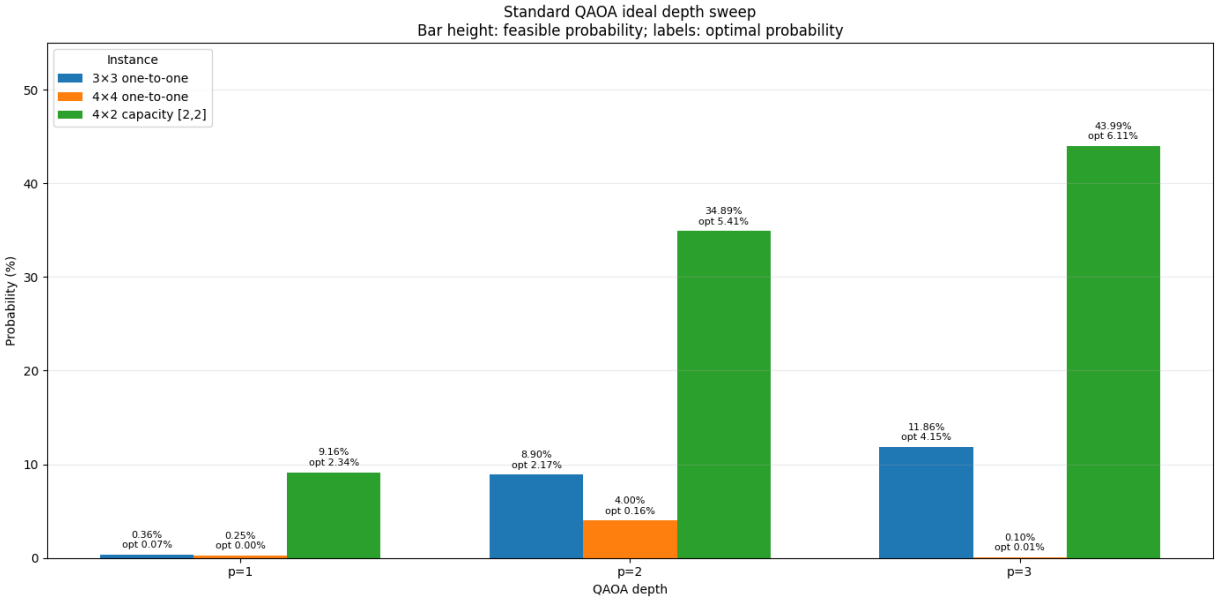

Figure 3: Ideal standard-QAOA comparison for $p = 1, 2, 3$. Bar heights show feasible probability and annotations show optimal-solution probability.

The capacitated 4×2 instance shows monotonic gains in $\langle Q \rangle$, P_{F}, and P_{opt}. The 3×3 instance increases P_{F} and P_{opt} through $p = 3$, although its best expected energy occurs at $p = 2$. The 4×4 instance is non-monotonic: $p = 2$ is the only tested depth that samples the optimum, while $p = 3$ returns to gap 0.28 and greatly reduces feasible mass. More layers therefore do not guarantee a better optimized state under a fixed optimizer budget.

Figure 4 shows the optimization histories for the original 3×3 instance.

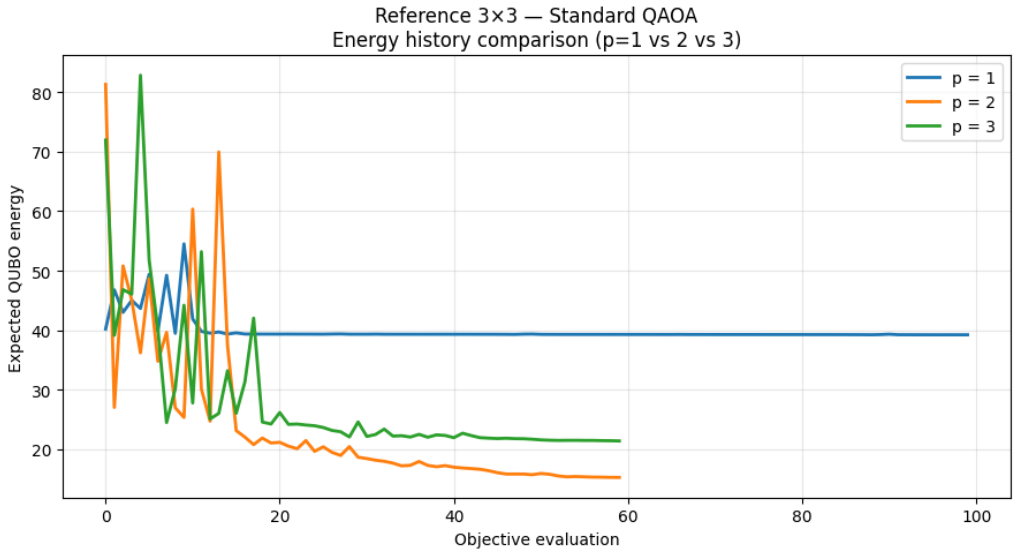

Figure 4: Expected-QUBO-energy histories for the reference 3×3 standard-QAOA runs.

7.3 Ideal GM-QAOA results

Table 5 confirms exact feasible-subspace evolution. State-preparation and full-circuit fidelities are numerically one. All three runs achieve zero best-score gap.

Table 5: Ideal $p = 1$ GM-QAOA results.

Instance	P_{F}	P_{opt}	Best score	Gap	$F(U_S)$	Circuit fidelity	Depth	Runtime (s)
3×3	1.000000	0.692281	2.70	0.00	1.000000	1.000000	1062	0.068595
4×4	1.000000	0.107441	3.08	0.00	1.000000	1.000000	20252	0.059578
4×2	1.000000	0.503666	3.25	0.00	1.000000	1.000000	395	0.086711

The ideal probability advantage of GM-QAOA is pronounced on the fixed cases, but the decomposed depths reveal the state-preparation cost. In particular, the 4×4 fixed construction has depth 20252 before any hardware routing experiment and was therefore not selected for QPU execution.

7.4 IBM Quantum results

The hardware experiment includes only the 4×2 instance and $p = 1$. Table 6 distinguishes best-sample quality from distribution concentration.

Table 6: Hardware quality for the exact-capacity 4×2 instance.

Algorithm	Shots	Minimum Q	Best score	Gap	P_{F}	P_{opt}
Standard QAOA	1024	-3.25	3.25	0.00	0.077148	0.015625
GM-QAOA	1024	-3.25	3.25	0.00	0.119141	0.028320

Both distributions contain `10011001`, the exact optimum. The most frequently observed standard-QAOA state is instead infeasible `10101101` with 23 counts. In GM-QAOA, the optimal state is the modal output with 29 counts, but the remaining probability is distributed broadly and mostly outside the feasible set.

7.5 Transpilation and resource overhead

Table 7: Logical and transpiled resource counts on `ibm_marrakesh`.

Algorithm	Depth before	Depth after	Size before	Size after	2Q before	2Q after	CZ	Transpile (s)
Standard QAOA	10	143	48	293	16	63	63	0.023831
GM-QAOA	11	1357	60	2180	12	496	496	0.132162

Relative to standard QAOA, the physical GM-QAOA circuit has 9.49× the depth, 7.44× the number of instructions, and 7.87× the native two-qubit gates. Relative to its own abstract circuit, GM-QAOA experiences 123.36× depth inflation, compared with 14.3× for standard QAOA.

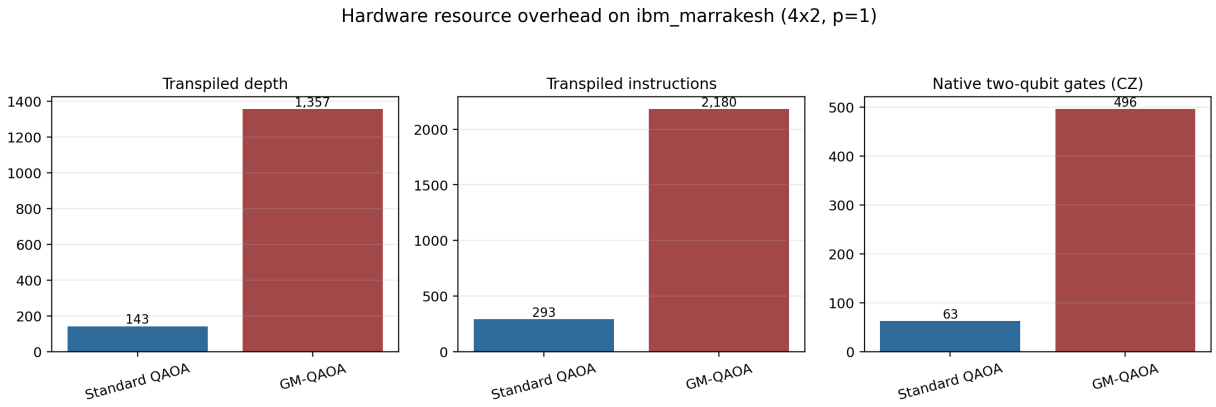

Figure 5: Transpiled resource overhead for the two circuits executed in the same Runtime job.

8 Error Analysis and Noise Characterization

8.1 Ideal versus hardware distributions

Figure 6 is the wide histogram generated in the executed notebook. It compares a 1024-shot ideal reference with the measured `ibm_marrakesh` counts on the same selected support. The standard distribution is moderately distorted. The GM-QAOA distribution exhibits a much larger collapse of the dominant ideal peaks and substantial population of states having zero ideal probability.

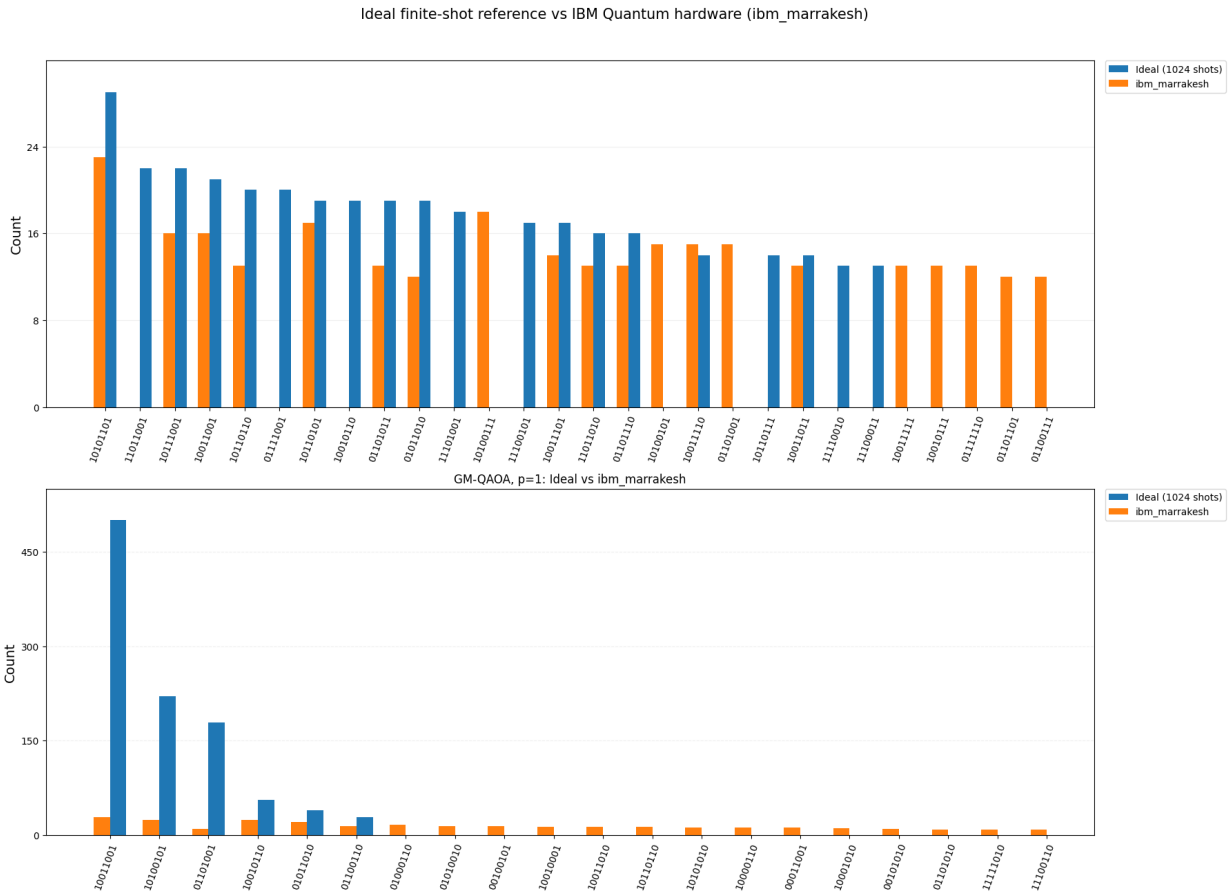

Figure 6: Ideal finite-shot reference versus IBM Quantum hardware for standard QAOA and GM-QAOA. The bottom panel makes the collapse of the ideal GM-QAOA peaks visible.

8.2 Feasible and optimal probabilities

Table 8 compares exact ideal values with complete hardware counts. The standard ideal values and its global distribution metrics are reconstructed within the six-decimal precision of the repository exports, as explained in Section 6.

Table 8: Ideal-to-hardware probability degradation.

Algorithm	Ideal P_{F}	QPU P_{F}	Ideal P_{opt}	QPU P_{opt}	ΔP_{opt}	Relative P_{opt} error
Standard QAOA	0.091582	0.077148	0.023380	0.015625	−0.007755	−33.17%
GM-QAOA	1.000000	0.119141	0.503666	0.028320	−0.475346	−94.38%

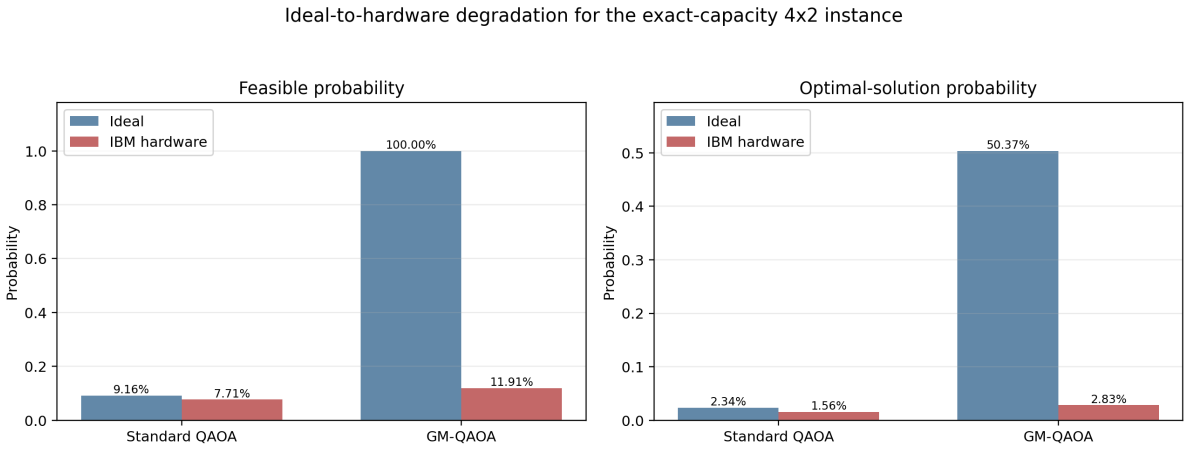

Figure 7: Ideal and physical feasible/optimal probabilities for the hardware instance.

The 95% Wilson intervals are

$$P^{\mathrm{std}}_{\mathrm{F,QPU}} \in [0.06234, 0.09512], \qquad P^{\mathrm{std}}_{\mathrm{opt,QPU}} \in [0.00964, 0.02523], \tag{41}$$

$$P^{\mathrm{GM}}_{\mathrm{F,QPU}} \in [0.10071, 0.14042], \qquad P^{\mathrm{GM}}_{\mathrm{opt,QPU}} \in [0.01979, 0.04038]. \tag{42}$$

The standard ideal optimum probability lies inside its physical interval, whereas the GM-QAOA ideal optimum probability is far outside the physical interval.

8.3 Global distribution metrics

Table 9: Distribution-level errors. Standard values marked † are reconstructed within the precision of the exported ideal probabilities.

Algorithm	TVD	Hellinger	F_{cl}	$1 - \mathrm{TVD}$
Standard QAOA†	0.2082	0.2346	0.8929	0.7918
GM-QAOA	0.8809	0.8284	0.0984	0.1191

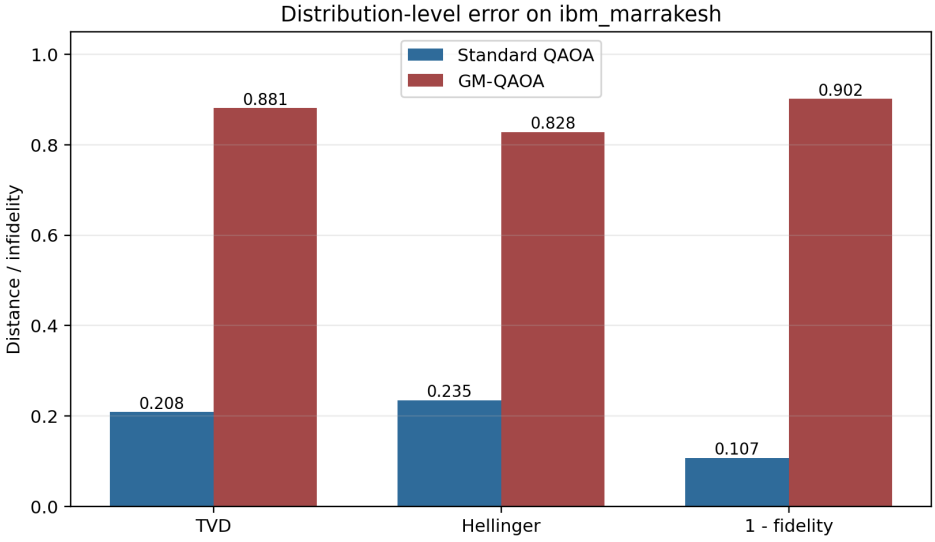

Figure 8: Distribution-level distance and infidelity measures derived from the complete archived counts.

For GM-QAOA, the ideal distribution has support only on $\mathcal{F}$. All six measured feasible probabilities remain below their ideal values, so the TVD equals the measured leakage:

$$\mathrm{TVD}_{\mathrm{GM}} = 1 - P^{\mathrm{GM}}_{\mathrm{F,QPU}} = 0.880859375. \tag{43}$$

This equality provides a direct interpretation of the error: 88.09% of measured probability has leaked outside the ideal support.

8.4 Energy distortion

The expected penalized energies change as follows:

Table 10: Ideal and observed expected QUBO energies.

Algorithm	Ideal $\langle Q \rangle$	QPU $\langle Q \rangle$	Difference
Standard QAOA†	18.3443	19.6276	+1.2833
GM-QAOA	−2.8453	20.2492	+23.0945

The GM-QAOA expected energy loses the advantage implied by its ideal feasible-state concentration, even though one optimal sample remains visible.

8.5 Noise mechanisms and evidence

The experiment does not isolate a single microscopic error channel. The observed distortion is consistent with the combined effects of:

- synthesis of controlled state preparations and the multi-controlled phase;
- routing on a sparse coupling graph;
- accumulated CZ and single-qubit gate error;
- decoherence during a substantially longer schedule;
- measurement error;
- calibration drift and stochastic variation across twirling realizations.

Dynamical decoupling and gate twirling were enabled, so the results already include those suppression techniques. Their presence does not constitute a no-mitigation control experiment. The resource contrast strongly supports circuit length as a major explanatory factor, but it does not prove that depth alone caused the observed difference.

8.6 Best-sample error versus statistical reliability

Both algorithms have zero best-sample optimization gap because the optimum appeared at least once. This statement is compatible with low reliability: the optimal state occurred only 16 times for standard QAOA and 29 times for GM-QAOA out of 1024 shots. A publication claim should therefore report the gap jointly with P_{opt}, confidence intervals, P_{F}, and global distances.

9 Discussion and Limitations

9.1 Depth is instance- and metric-dependent

The ideal standard-QAOA sweep does not support a universal claim that larger p improves performance. For the 4×2 instance, $p = 3$ is best under expected energy, feasible probability, and optimal probability. For 3×3, $p = 2$ has the lowest expected energy while $p = 3$ has higher feasible and optimal probabilities. For 4×4, the tested $p = 3$ run is worse than $p = 2$ and fails to recover the optimum. Optimization budget, initialization, and landscape structure interact with depth.

9.2 GM-QAOA shifts rather than removes complexity

In ideal simulation, GM-QAOA offers a clean separation between feasibility and optimization. Penalty terms are unnecessary inside $\mathcal{F}$, feasible probability is exactly one, and the optimum receives substantially more probability than under standard QAOA at $p = 1$. The hardware result shows the trade-off emphasized by Bärtschi and Eidenbenz: the approach is useful only when U_S and its inverse can be compiled efficiently [3].

For the compact 4×2 case, the physical Grover-mixer circuit is nearly ten times deeper and contains nearly eight times as many native two-qubit gates as the standard circuit. Its ideal 50.37% optimum probability collapses to 2.83%. Thus, feasible-subspace design at the logical level is not sufficient; state-preparation architecture and device-aware synthesis determine whether the theoretical benefit survives.

9.3 Alignment with QI26-20

The artifact fulfills the central implementation path of QI26-20: synthetic assignment data, one-to-one and one-to-N constraints, QUBO formulation, classical baselines, ideal QAOA, real IBM Quantum execution, and analysis of depth and noise-related overhead [1]. It also preserves the depth supplement's restricted $p = 1, 2, 3$ experiment [4]. A complete six-week research program would additionally require a systematic size family, broader tuning, and repeated physical experiments.

9.4 Alignment with Bärtschi and Eidenbenz

The notebook implements the operational equations

$$U_S|0\rangle = |F\rangle, \qquad U_P(\gamma) = e^{-i\gamma H_C}, \qquad U_M(\beta) = U_S D_0(\beta) U_S^\dagger \tag{44}$$

and verifies ideal feasible-subspace preservation. It does not implement the paper's general permutation generator or demonstrate its asymptotic complexity. The absence of Hamiltonian-simulation error in the logical mixer must not be confused with absence of physical noise.

9.5 Threats to validity

The main limitations are:

1. **Small fixed instances.** Eight, nine, and sixteen logical qubits do not establish scaling behavior.
2. **Single deterministic optimization.** No optimizer-seed variance or parameter-sensitivity distribution is measured.
3. **One physical job.** The study cannot estimate temporal drift or between-job variance.
4. **One backend.** Results are not portable to other topologies or calibration regimes without new transpilation and execution.
5. **Hardware depth restricted to $p = 1$.** The ideal $p = 2, 3$ conclusions are not physical results.

6. **Fixed feasible-state preparations.** They are exact demonstrations, not a general scalable library.
7. **No ablation study.** Dynamical decoupling and twirling are enabled, but there are no matched controls without these options.
8. **Standard global metric reconstruction.** Standard ideal-to-hardware distances use a distribution reconstructed within the six-decimal precision of repository exports; the complete ideal vector should be exported directly in future releases.

9.6 No quantum-advantage claim

Exact and Hungarian solvers obtain the optimum orders of magnitude faster than the quantum workflow at these sizes. The reported contribution is a reproducible formulation and hardware-error benchmark. Neither runtime advantage nor solution-quality advantage over appropriate classical algorithms is demonstrated.

10 Conclusion and Future Work

This study delivers a traceable QUBO–QAOA benchmark for generalized bipartite assignment. The formulation supports one-to-one assignment and fixed exact capacities, and its QUBO-to-Ising mapping is numerically validated. Four classical baselines, a standard-QAOA depth sweep, ideal GM-QAOA, backend-specific transpilation, and a real IBM Quantum comparison are integrated in one repository artifact.

The ideal standard-QAOA study shows that the effect of depth is not monotonic across instances. The exact-capacity 4×2 case benefits consistently through $p = 3$, while the 4×4 case is best at $p = 2$ under the tested settings. Ideal GM-QAOA preserves the feasible subspace and yields zero best-score gap for all three fixed instances.

On `ibm_marrakesh`, both $p = 1$ algorithms sampled the exact optimum. Nevertheless, standard QAOA retained a distribution much closer to its ideal reference than GM-QAOA. The Grover-mixer circuit incurred depth 1357 and 496 CZ gates, versus depth 143 and 63 CZ gates for standard QAOA. Its feasible mass fell from 100% to 11.91%, and its optimal probability from 50.37% to 2.83%. The primary scientific conclusion is therefore not that GM-QAOA fails as an ideal ansatz, but that the specific fixed feasible-state preparation and selective-phase implementation are too costly to preserve their ideal advantage on this hardware execution.

Future work should:

1. export complete ideal and measured distributions directly with every run;
2. repeat matched jobs across time and backends, with uncertainty over calibrations;
3. evaluate hardware $p = 2, 3$ only after resource preflight;
4. replace generic or controlled state preparations with structured W/Dicke, bitmask, swap, and ancilla-reuse constructions;
5. compare alternative multi-controlled-phase syntheses and layouts;
6. perform ablations for readout mitigation, dynamical decoupling, twirling, and postselection;
7. construct a controlled family of instance sizes and capacity vectors;
8. report total resource cost jointly with solution quality and classical baselines.

Within its stated scope, the artifact satisfies the intended Phase-1 objective: it converts a student-developed assignment benchmark into a reproducible research workflow with explicit theoretical alignment, real-hardware evidence, error analysis, and clearly bounded claims.

Author Contributions

The contribution statement follows the ownership recorded in the notebook. Jiya Maurya led the mathematical formulation and QUBO derivation, with support from Satkar Juneja and Ilia Fazeli. Qyngshuq and Abhishek Raj led synthetic-data generation and constraint validation, with support from Jiya Maurya and Satkar Juneja. Satkar Juneja led the classical baselines and evaluation metrics, with support from Jiya Maurya and Marcos Jiménez. Marcos Jiménez led the Qiskit/QAOA simulator implementation, with support from Ilia Fazeli, Qyngshuq, Abhishek Raj, and Satkar Juneja. Ilia Fazeli led documentation and experiment tracking, with support from Marcos Jiménez and Jiya Maurya. Iván Barrientos Salas provided conceptualization, supervision, scientific auditing, integration of generalized capacities and GM-QAOA, hardware methodology, validation, and manuscript preparation. All authors should confirm the final CRediT taxonomy and author order before journal submission.

Data and Code Availability

The reproducible artifact is organized as the repository `QI26-20-QUBO-QAOA-Benchmark`. It contains the executed notebook, dependency specification, exported result tables, scientific-claims checklist, and compliance matrix. The archived IBM Quantum job identifier is `d9vl8qfo3ppc73akfak0`. The public repository URL should be inserted when the repository is released. The full distribution table reconstructed from the archived Runtime result is included with the manuscript source under `Data/full_hardware_distributions.csv`.

Acknowledgments

This work was developed in the QI26-20 / QIntern 2026 research context. The authors acknowledge the use of Qiskit and IBM Quantum services. Access to the quantum processing unit does not imply endorsement of the conclusions by IBM.

Funding and Conflicts of Interest

Funding and conflict-of-interest declarations were not specified in the repository artifacts and must be confirmed by all authors before external submission.

A Compliance Matrix

Table 11: Formal compliance with QI26-20, the depth supplement, and Bärtschi–Eidenbenz.

Reference	Requirement	Status	Evidence and reservation
Depth supplement	Preserve original 3×3, $p = 1$ experiment	**Met**	The reference run remains explicit and uses the original initialization and analysis aliases.
Depth supplement	Add 4×4 one-to-one instance	**Met**	Reproducible seed-2026 affinity matrix and exact one-to-one constraints.
Depth supplement	Add 4×2 capacities $[2, 2]$	**Met**	Exact squared capacity penalty and direct feasibility validation.
Depth supplement	Standard-QAOA sweep $p = 1, 2, 3$	**Met ideally**	Nine deterministic statevector experiments; no additional depth grid.
QI26-20	Classical baselines and same-instance comparison	**Met**	Exact, Hungarian, greedy, and simulated annealing use the same matrices and score definitions.
QI26-20	Real hardware execution	**Met for fixed case**	Standard QAOA and GM-QAOA executed on `ibm_marrakesh`, job `d9vl8qfo3ppc73akfak0`.
QI26-20	Depth, noise, and resource analysis	**Partial**	Ideal depth sweep and $p = 1$ QPU resource/error analysis are present; no QPU $p = 2, 3$ campaign.
QI26-20	Scaling over increasing graph sizes	**Partial**	Three sizes are represented, but no systematic scaling regression or broad family.
Bärtschi–Eidenbenz	Equal feasible-state superposition	**Met for fixed cases**	State-preparation fidelity equals one in ideal simulation.
Bärtschi–Eidenbenz	Parameterized Grover mixer	**Met operationally**	$U_S D_0(\beta) U_S^\dagger$ is implemented with a selective phase.
Bärtschi–Eidenbenz	Ideal feasible-subspace preservation	**Met ideally**	$P_{\mathrm{F}} = 1$ and full-circuit state fidelity equals one for all fixed cases.
Bärtschi–Eidenbenz	General $O(n^3)$ permutation preparation	**Not implemented**	Phase 1 uses exact fixed-size constructions and makes no general asymptotic claim.
Scientific claim	Quantum advantage	**Not established**	Classical exact and Hungarian methods dominate these small instances; one hardware job is insufficient.

B Transpiled Circuit Views

The complete folded standard-QAOA ISA circuit is shown in Figure 9. Idle physical wires are omitted by the notebook drawer. The GM-QAOA circuit is substantially taller in folded form, so it is split into four overlapping panels. The full original image is retained as `Figures/gm_qaoa_transpiled_folded_full.png` in the source package.

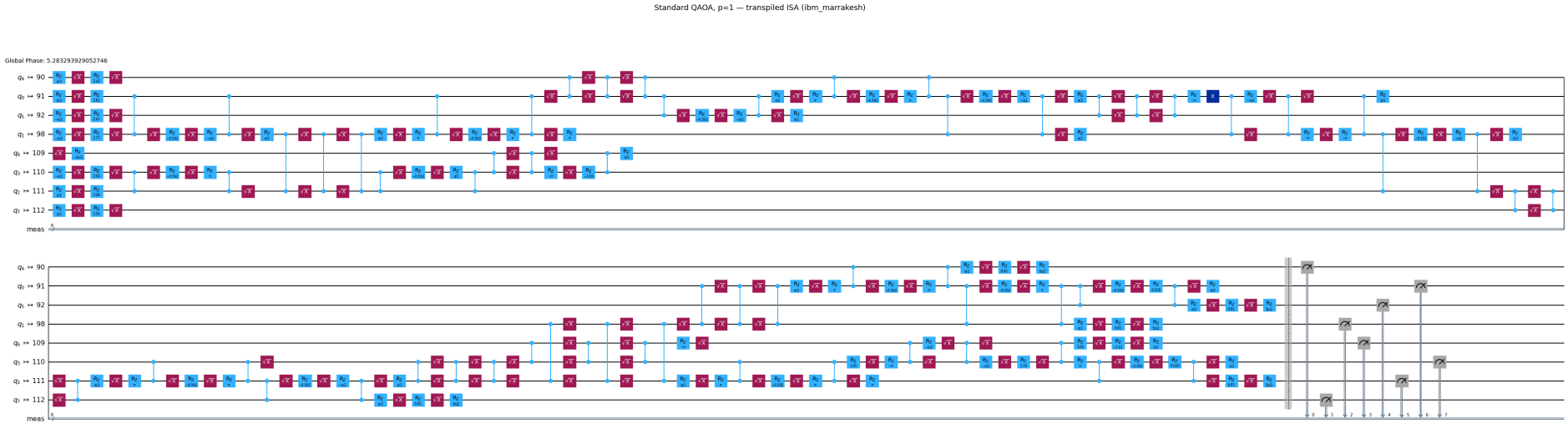

Figure 9: Standard-QAOA $p = 1$ circuit after transpilation to `ibm_marrakesh`.

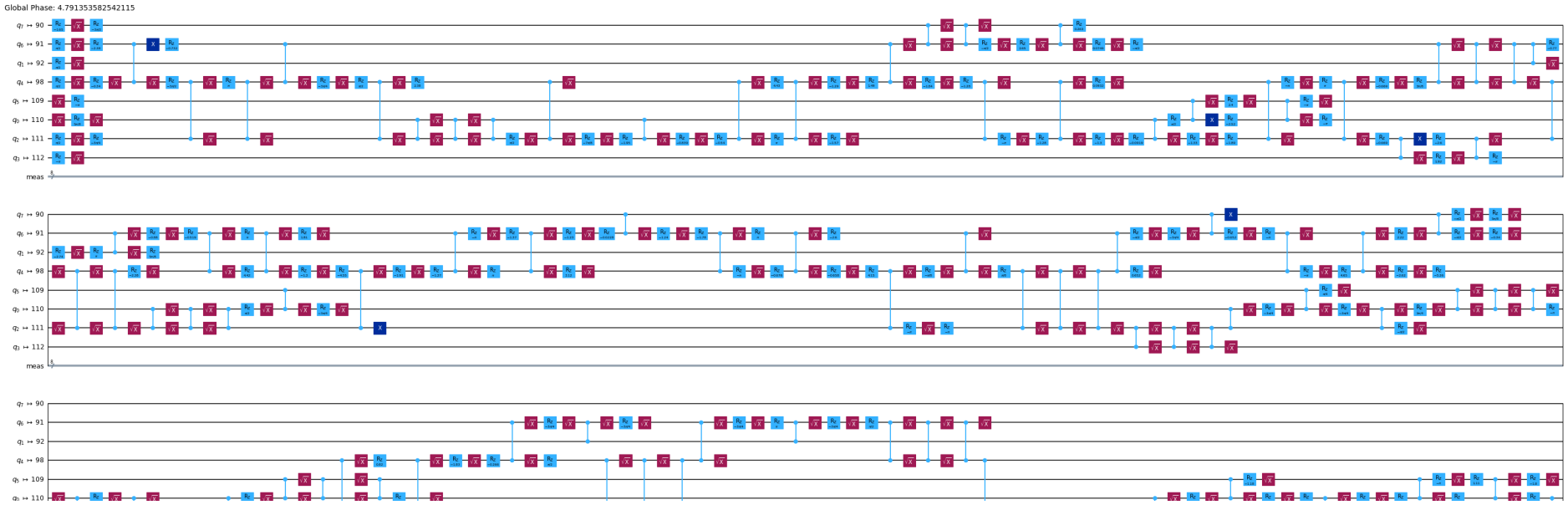

Figure 10: GM-QAOA $p = 1$ transpiled circuit, folded-view panel 1 of 4.

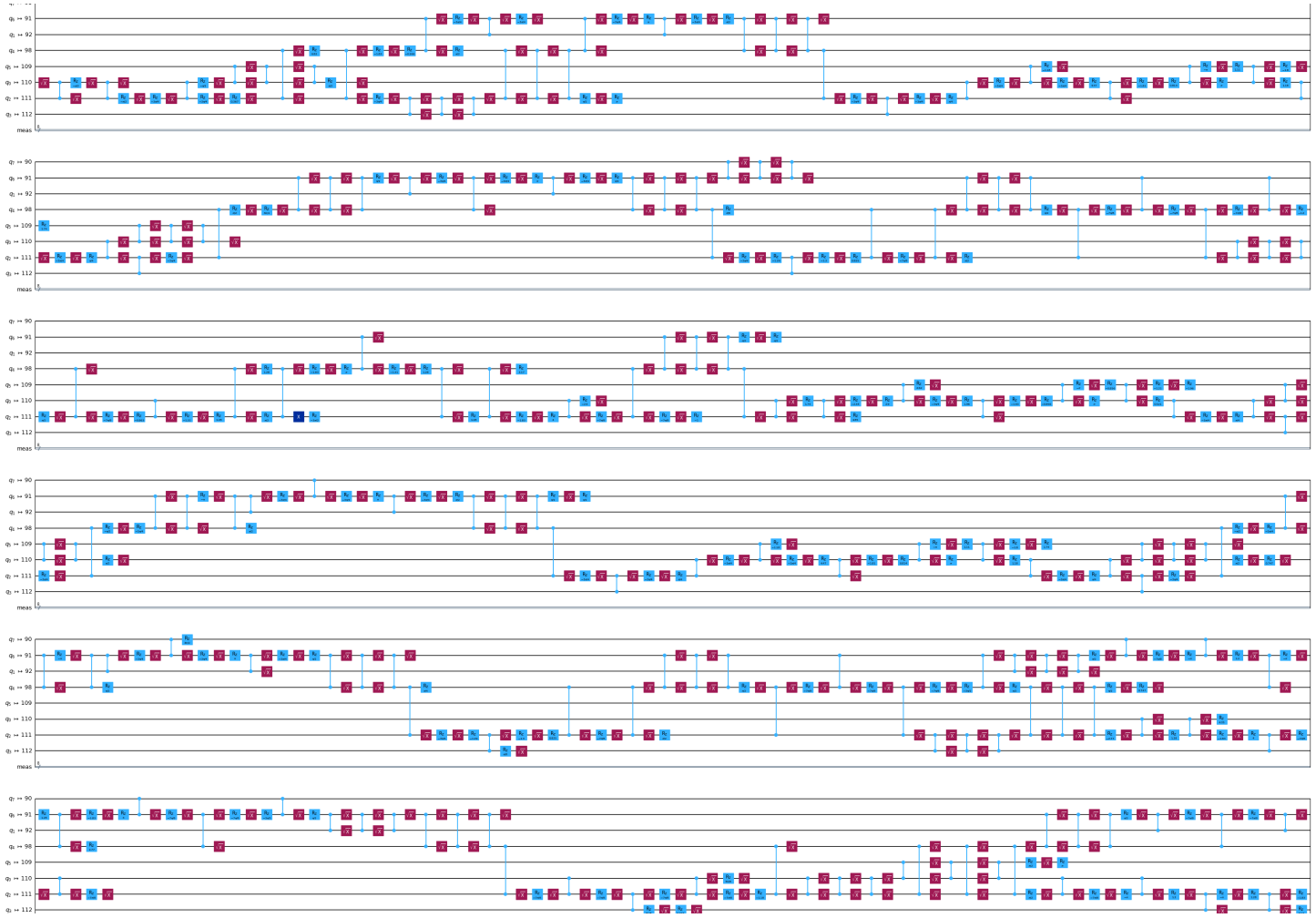

Figure 11: GM-QAOA $p = 1$ transpiled circuit, folded-view panel 2 of 4.

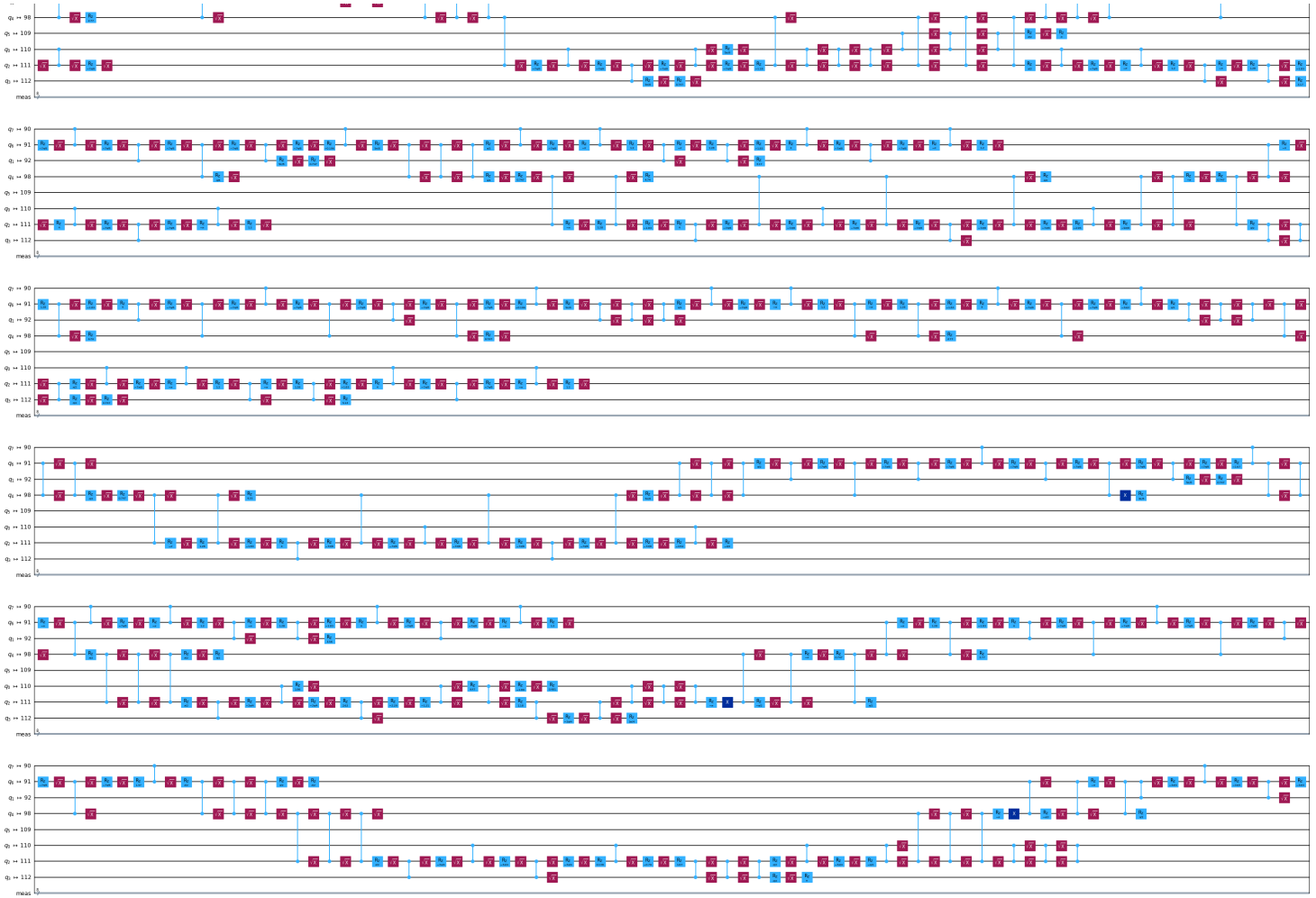

Figure 12: GM-QAOA $p = 1$ transpiled circuit, folded-view panel 3 of 4.

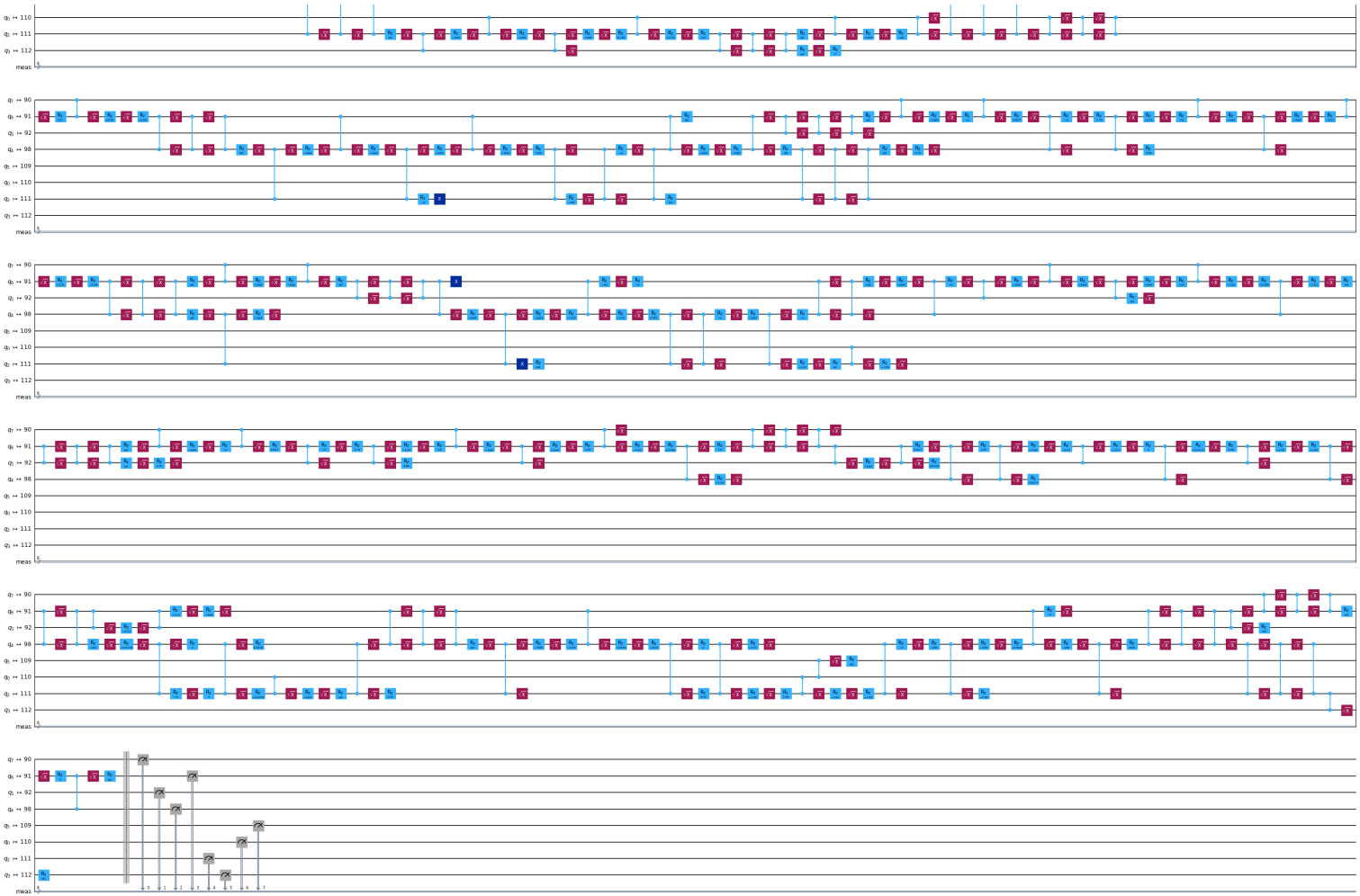

Figure 13: GM-QAOA $p = 1$ transpiled circuit, folded-view panel 4 of 4.

C Hardware Distribution Detail

C.1 Feasible-state counts

Table 12: Complete counts on the six feasible states of the 4×2 problem.

	Standard QAOA		GM-QAOA		
Bitstring	Count	Ideal probability	Count	Ideal probability	Score
10011001	16	0.023380	29	0.503666	3.25
10100101	15	0.016493	24	0.195717	2.60
01101001	15	0.016240	10	0.174404	2.55
10010110	11	0.013311	24	0.054315	2.20
01011010	12	0.013112	21	0.042262	2.15
01100110	10	0.009046	14	0.029636	1.50
Total feasible	79	0.091582	122	1.000000	–

C.2 Most frequent outcomes

Table 13: Ten most frequent outcomes per hardware circuit.

Algorithm	Bitstring	Count	Observed $q(x)$	Ideal $p(x)$	Score	Feasible
Standard	10101101	23	0.022461	0.019028	3.50	No
Standard	10100111	18	0.017578	0.014731	2.90	No
Standard	10110101	17	0.016602	0.015252	3.30	No
Standard	10111001	16	0.015625	0.019959	3.80	No
Standard	10011001	16	0.015625	0.023380	3.25	Yes
Standard	10011110	15	0.014648	0.016861	3.10	No
Standard	01101001	15	0.014648	0.016240	2.55	Yes
Standard	10100101	15	0.014648	0.016493	2.60	Yes
Standard	10011101	14	0.013672	0.016719	3.65	No
Standard	10011011	13	0.012695	0.017883	3.55	No
GM	10011001	29	0.028320	0.503666	3.25	Yes
GM	10010110	24	0.023438	0.054315	2.20	Yes
GM	10100101	24	0.023438	0.195717	2.60	Yes
GM	01011010	21	0.020508	0.042262	2.15	Yes
GM	01000110	16	0.015625	0.000000	0.95	No
GM	00100101	14	0.013672	0.000000	1.80	No
GM	01010010	14	0.013672	0.000000	1.25	No
GM	01100110	14	0.013672	0.029636	1.50	Yes
GM	10010001	13	0.012695	0.000000	2.35	No
GM	10011010	13	0.012695	0.000000	2.70	No

The complete 512-row-by-two-algorithm table is supplied as a CSV rather than reproduced in full in the PDF.

D Reproducibility Metadata and Claims Boundary

D.1 Repository artifact

The source repository contains:

```
QI26-20-QUBO-QAOA-Benchmark/
|-- README.md
```

```
|-- requirements.txt
|-- QI26_20_QUBO_QAOA_GM_QAOA_Benchmark.ipynb
|-- results/
`-- paper/
```

The manuscript PDF is intended for the repository path

`paper/QI26_20_preprint.pdf`

D.2 Core run parameters

Table 14: Core reproducibility parameters.

Parameter	Value
Data / transpiler / simulator seed	2026
Standard depths	$p \in \{1, 2, 3\}$ ideal; $p = 1$ hardware
GM depth	$p = 1$
Optimizer	COBYLA
Ideal candidate shots	4096
Hardware shots	1024 per circuit
Hardware instance	4×2, exact capacities $[2, 2]$
Backend	`ibm_marrakesh`
Optimization level	3
Dynamical decoupling	XpXm
Gate twirling	Enabled
Job ID	`d9vl8qfo3ppc73akfak0`

D.3 Supported and unsupported statements

Supported by the artifact	Not supported by the artifact
One-to-one and fixed exact-capacity QUBO models are implemented and validated.	The implementation supports arbitrary dynamic capacities without extension.
Standard QAOA is compared at $p = 1, 2, 3$ in ideal simulation.	Standard QAOA was run on hardware at all three depths.
The fixed GM-QAOA circuits preserve the feasible subspace ideally.	The QPU preserves the GM-QAOA feasible subspace.
Both physical circuits sampled the exact optimum.	Either algorithm finds the optimum with high probability.
The operational Grover mixer matches the equations in Bärtschi–Eidenbenz.	The general $O(n^3)$ state-preparation construction is reproduced.
The recorded GM-QAOA circuit has substantially higher depth and CZ count.	The same ratio holds for all devices and instances.
No quantum advantage is established.	The data establish or refute quantum advantage in general.

D.4 Credential hygiene

No IBM Quantum credential is required to inspect or reproduce the archived analysis. Any future execution should obtain the token from a Colab secret or environment variable and must not commit the token to the repository.

References

[1] Iván Barrientos Salas. QI26-20: Quantum computing for combinatorial assignment optimization. Technical report, QIntern / QWorld, 2026. Internal project brief.

[2] Stuart Hadfield, Zhihui Wang, Bryan O'Gorman, Eleanor G. Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms*, 12(2):34, 2019.

[3] Andreas Bärtschi and Stephan Eidenbenz. Grover mixers for QAOA: Shifting complexity from mixer design to state preparation. In *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 72–82, 2020.

[4] QI26-20 Research Team. Depth extension: Generalized capacities and QAOA depth sweep. Internal files depth.pdf and depth.ipynb, 2026.

[5] Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

[6] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[7] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

[8] Qiskit contributors. Qiskit: An open-source framework for quantum computing. Software, version 2.5.1, 2026.

[9] Pauli Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020.

[10] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.